FEMU CDFTL Report

Storage Architecture, Team 1

2021310821 정현일

2021310325 오동현

Translation Block

해당 과목 Lab1에서 구현했었던 nand simulator의 코드를 바탕으로 1 chip, 1 plan, 1 bank, 16 block의 16MB의 nand emulator를 `ftl.c` 에 구현하였다.

과제 FEMU의 configuration과 동일하게 block하나 당 page 수는 256개, page 하나의 크기는 4KB를 갖도록 하였으며, translation block전체를 `tnand` 라는 이름의 포인터로 관리한다.

해당 emulator가 제공하는 api는 `tnand_init()` , `tnand_write()` , `tnand_read()` 이며, 낸드 동작이 성공할 경우 0을 반환, 실패할 경우 각 케이스에 해당하는 에러코드를 반환한다.

```
#define NAND_SUCCESS       0
#define NAND_ERR_INVALID    -1
#define NAND_ERR_OVERWRITE -2
#define NAND_ERR_POSITION  -3
#define NAND_ERR_EMPTY     -4
```

Fetch-in

PMT를 read하는 `get_maptbl_ent()` 에서 cahce miss가 발생하는 경우에 호출되는 `fetch_in()` 와, PMT를 write하는 `set_maptbl_ent()` 에서 `replace()` 과정 중, update하는 해당 dlpn이 cache에 존재하지 않을 때, CTP entry만을 fetch하는 `ctp_fetch_in()` 을 별도로 구현하였다.

fetch_in()

1. cache가 full인지에 대한 여부를 `is_cache_full()` 을 호출하여 확인한다.
    - 초기 설정으로 cache full 기준을 1024*8 + 32*4096 bytes로 설정하였다.
        - 1024는 초기 CMT entry 설정 수임.
        - 8은 CMT entry 하나 당 size임.
        - 32는 초기 CTP entry 설정 수임.
        - 4096은 CTP entry 하나 당 size임.
    - 현재 cache에 상주하고 있는 CMT entries와 CTP entries가 차지하고 있는 bytes 수와 fetch 될 CMT 1 entry, CTP 1 entry 만큼의 bytes 수의 합이 cache full 기준 보다 크다면 cache가 full 인 것으로 판단한다.
2. cache가 full이라면, 논문에서와 같은 정책으로 CMT 1 entry, CTP 1 entry 만큼의 공간을 확보한다.
    - 우선 LRU 정책으로 CMT entry 하나를 방출한다.
    - 그 후, CMT에 있는 entries 중, 각 매핑이 CTP에도 존재하는 entries의 수가 CTP에 포함할 수 있는 entries 수 이상만큼 존재한다면, 해당 mapping을 CMT로 flush 후, 방출한다.
    - 만약 그 만큼의 CMT entry가 없다면, LRU 정책으로 CTP entry 하나를 방출한다.
3. cache에 공간이 확보된 후, 요청된 dlpn에 해당하는 translation page를 `tnand_read()` 후, 요청된 mapping은 CMT entry로 생성하여 테이블에 삽입하고, 읽은 1 page를 CTP entry로 만들어 테이블에 삽입한다.

ctp_fetch_in()

전체적으로 `fetch_in()` 과 동일한 알고리즘이지만, CMT entry를 생성하지 않고, 그에 따라 CMT entry 하나 만큼의 공간도 확보하는 로직을 제거한 버전이다.

Replace

`replace()`는 data가 NAND에 쓰여지는 경우 DPPN을 update 할 때 쓰이는 함수로 크게 4가지 경우를 나눠서 동작하도록 구현하였다.

1. CTP와 CMT 모두 Hit된 경우이다. 이 경우에는 CTP의 해당 부분을 새로운 DPPN으로 수정하고 CMT 중 바꿀려는 TLPN에 포함되어있는 entry를 모두 CMT에서 제거한다. 수정한 부분은 lru list에서 제일 앞으로 옮긴다.
2. CTP는 Hit CMT는 Miss인 경우이다. 이 경우에는 CTP의 해당 부분만 새로운 DPPN으로 수정하고 해당 GTD entry의 dirty bit를 set한다. 이 경우에도 수정한 부분은 lru list에서 제일 앞으로 옮긴다.
3. CTP는 Miss, CMT는 Hit된 경우이다. 이 경우에는 CMT의 해당 부분만 새로운 DPPN으로 수정하고 해당 CMT entry의 dirty bit를 set한다. 수정한 CMT entry는 lru list에서 제일 앞으로 옮긴다.
4. CTP와 CMT 모두 Miss인 경우이다. 이 경우에는 수정하려는 entry가 포함된 Translation Page를 CTP에만 올리고 해당 entry를 수정한다. fetch_in은 CMT와 CTP에 모두 올리지만 이 경우에는 CMT에만 올리기 때문에 fetch_in 함수에서 CMT에 올리는 부분을 제거한 CTP_fetch_in 함수를 새로 만들어서 관리하였다.

get_maptbl_ent()

1. CMT hit을 확인 후, 요청된 mapping이 CMT에 존재하는 entry일 경우, 해당하는 dppn을 반환한다.
2. CMT miss인 경우, CTP hit을 확인 후, 요청된 mapping에 대응하는 Translation page가 CTP에 존재하는 entry일 경우, 해당하는 mapping을 찾아 dppn을 반환한다.
3. Cache miss인 경우, `fetch_in()` 을 호출하여 요청된 mappin을 CMT, CTP에 모두 올린 후, 해당하는 dppn을 반환한다.

set_maptbl_ent()

`set_maptbl_ent()`는 원래 FTL에서는 mapping table의 dppn을 부분을 수정하는 함수였다. 그래서 위에서 설명한 `replace()`를 이용하여 translation page에 dppn을 수정하도록 하였다.

Data GC

기존 FEMU FTL은 GC를 할 때 victim block을 돌면서 valid page가 있을 때마다 `gc_write_page()` 함수에서 valid copy를 진행하였고 `set_maptbl_ent()`로 ppa를 update 해줬다.

CDFTL 논문에서는 GC가 캐시 entry를 evict하지 않도록 CMT와 CTP에 없으면 캐시에 올리지 않고 translation page를 읽어와 수정하고 바로 write를 하도록 했다. 해당 알고리즘을 `set_maptbl_datagc()`에서 ppa를 update하도록 수정하였다.

Victim block의 각 valid page마다 진행

1. 먼저 CMT에 있는지 확인한 후 있으면 CMT를 수정하였다.
2. CTP를 확인하고 있으면 CTP를 수정하였다.
3. CMT와 CTP에 없을 경우에는 해당 translation page를 읽어와서 수정한 후 바로 write 해줬다.

Map GC

Translation page write가 발생하는 경우 trigger 조건을 확인하여 `map_garbage_collection()` 를 통해 translation block에 대한 garbage collection을 수행한다.

Trigger조건을 확인하는 부분

- dirty CMT entry를 방출할 때, 대응하는 page가 CTP에 캐시되어있지 않아 nand로 flush하는 경우
- dirty CTP entry를 방출할 때, nand로 flush하는 경우
- Data GC에서 valid copy를 진행할 때, nand write가 발생하는 경우

1. Greedy policy로 invalid page가 가장 많은 block을 victim으로 선정한다.
2. 선정된 block에서 valid page가 있을 경우
    a. CTP에 해당 페이지가 캐시되어 있으면서 dirty 상태인 경우, 해당 CTP entry를 copy

b. CTP에 해당 페이지가 없는 경우 해당 페이지를 nand에서 read

c. 해당 페이지 내의 각 mapping이 dirty상태로 CMT에 캐시되어 있는 경우 해당 mapping을 copy

d. 캐시된 dirty update를 모두 반영 후, ditry bit를 unset후, nand에 write

3. 모든 valid page를 copy한 후, 해당 victim block을 erase

Cache Hit/Miss

`struct ssd` 구조체 안에 `cmt_hits`, `ctp_hits`, `cache_misses` 와 `total_requests` 변수를 추가해

`get_maptbl_ent()` 와 `set_maptbl_ent()` 에서 `total_requests` 를 증가시켜 주었고 `cmt_find_entry()` 와 `ctp_find_entry()` 가 NULL이 아닐 때마다 각각의 hit를 count, 둘 다 miss인 경우에는 `cache_misses` 를 증가시켜주었다. 출력은 WAF를 출력하는 bb_get_log 함수를 수정해서

`sudo nvme write-amplification-log /dev/nvme0n1` 을 입력하면 출력하게 구현하였다.

개선한 점(Data GC)

논문의 Data GC는 valid page의 l2p mapping이 CMT와 CTP에 모두 없을 때마다 nand_read와 nand_write를 한다. 만약 한 block에 같은 translation page에 속하는 valid page가 많다면 overwrite 횟수가 굉장히 많아진다. 그래서 한 block에 대해 먼저 valid page들의 mapping(dlpn, dppn)을 저장하고 block 전체를 확인한 후에는 같은 tvpn에 해당하는 매핑들은 메모리에서 한 번에 수정한 후, 한 번만 write하여 overwrite의 횟수를 줄였다.

기존의 코드에서는 valid page가 있으면 `gc_write_page()` 에서 써주고 ppa를 수정하므로, 개선 방법은 같은 함수인 `gc_write_page()` 에서 valid page들의 new mapping(dlpn, dppn)을 bucket_pool에 tvpn을 기준으로 저장하였다. 그리고 저장해둔 정보를 바탕으로 `clean_one_block()` 에서 모든 page를 확인 후에 `set_maptbl_batchgc()` 를 통해 한 번에 `tnand_read()` 해와서 update 후에 한 번에 `tnand_write()` 하여 같은 tvpn에 대해서 overwrite가 일어나지 않게 했다.

Implementation

```
typedef struct _Bucket {
    uint64_t dlpn;
    uint64_t dppn;
    struct _Bucket* next;
} Bucket;

struct _TvpnBucket{
    Bucket* gc_copy_list[MAX_TVPN+1];
    uint64_t exist[MAX_TVPN]; // 특정 tvpn에 대한 gc_copy_list의 idx
    uint64_t bucket_size;
    uint64_t pool_idx;
};

Bucket bucket_pool[256]; // NPAGES_PER_BLK (pages per block) 크기
struct _TvpnBucket TvpnBucket;
```

하나의 mapping은 `_Bucket` 구조체에 dlpn, dppn을 통해 저장하며, block 하나 당 페이지 크기만큼의 `bucket_pool` 배열을 통해 매핑하나가 생길 때마다 일시적으로 저장할 공간을 할당 받는다.

`_TvpnBucket` 구조체 타입인 `TvpnBucket`

- `gc_copy_list` : victim Block에 존재하는 valid page의 tvpn별로 하나의 element를 차지하며, 각 element는 해당 tvpn의 Bucket에 대한 linked list head이다.
- `exist` : victim Block에서 맞이하는 valid page의 tvpn을 index로 삼으며 각 tvpn이 할당된 순서대로 1, 2, 3... 를 저장하고 해당 순서는 gc_copy_list의 index이다. 즉 Tvpn Linked list의 key를 담당하는 map 역할을 한다.
- `bucket_size` : `TvpnBucket` 이 총 몇개의 tvpn을 담당하고 있는지를 나타냄
- `pool_idx` : valid page의 수를 나타내며, 각 valid page가 발견될 때마다 1씩 증가되어 `gc_copy_list` 의 index를 담당한다.

```
void set_maptbl_batchgc(struct ssd *ssd)
{
    for(int i = 1; i <= TvpnBucket.bucket_size; i++) {
        if(tblk_curr.p_cur >= NPAGES) {
            tblk_curr = (BNUM){dequeue(fblk_list), 0};
            insert_minheap(ablk_list, tblk_curr.bnum);

            if (fblk_list->q_size < 1) { // map gc
                map_garbage_collection();
            }
        }

        Bucket* curr = TvpnBucket.gc_copy_list[i];
        uint64_t tvpn = curr->dlpn / NUM_MAPPINGS_PER_PAGE;
        uint64_t old_tppn = gtd[tvpn].tppn.ppa;
        uint64_t data[NUM_MAPPINGS_PER_PAGE];

        if (old_tppn != INVALID_PPN) {
            uint64_t old_blk = old_tppn / NPAGES;
            uint64_t old_page = old_tppn % NPAGES;
            tp2l[old_tppn] = INVALID_PPN;
            blk_invalid[old_blk]++;
            tnand_read(0, old_blk, old_page, data);
        } else memset(data, 0xff, PAGE_DATA_SIZE);


        int write_flag = 0;
        struct ctp_entry* ctp_curr = ctp_find_entry(tvpn);
        while (curr) {
            uint64_t lpn = curr->dlpn;
            struct cmt_entry* cmt_curr = cmt_find_entry(lpn);

            if(cmt_curr != NULL) {
                cmt_curr->data.dppn.ppa = curr->dppn;
                cmt_curr->data.dirty = true;
            } else if(ctp_curr != NULL) {
                ctp_curr->mp->dppn[lpn % NUM_MAPPINGS_PER_PAGE].ppa = curr->dppn;
                gtd[tvpn].dirty = true;
            } else {
                data[lpn % NUM_MAPPINGS_PER_PAGE] = curr->dppn;
                write_flag = 1;
            }
            curr = curr->next;
        }

        if(write_flag) {
            uint64_t blk = tblk_curr.bnum;
            uint64_t page = tblk_curr.p_cur++;
            uint64_t tppn = blk*NPAGES + page;
            gtd[tvpn].tppn.ppa = tppn;
            tp2l[tppn] = tvpn;

            tnand_write(0, blk, page, data);
            ssd->tnand_writes_gc++;
```

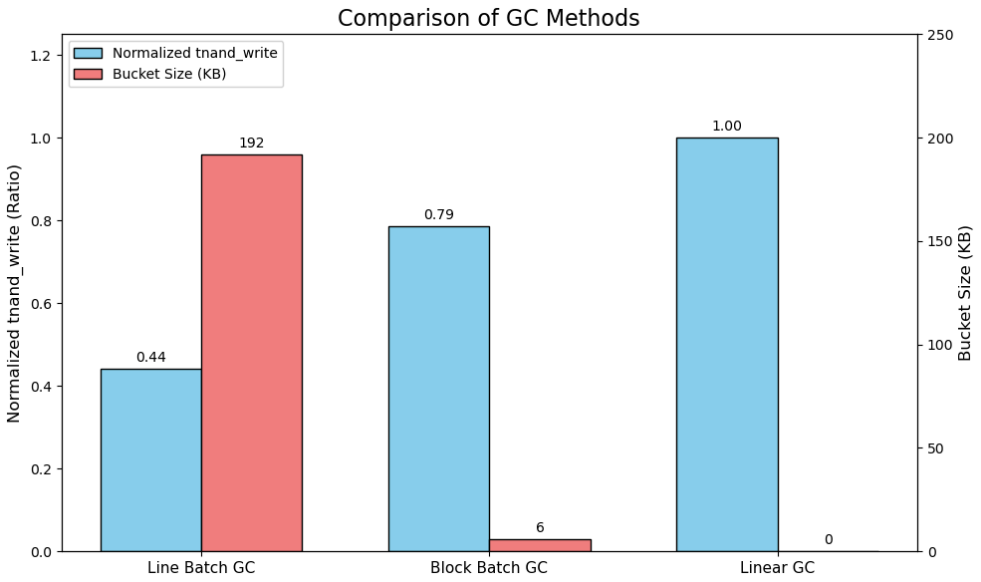
```
        } else {
            gtd[tvpn].tppn.ppa = INVALID_PPN;
        }
    }
    for(int i = 1; i <= TvpnBucket.bucket_size; i++) TvpnBucket.exist[(TvpnBucket.gc_copy_list[i]→dlpn) / NUM_
MAPPINGS_PER_PAGE] = 0;
}
```

앞 문단에서 설명한 알고리즘에 대한 구현이다. 해당 함수는 `clean_one_block()` 함수에서 한 block에 대한 모든 valid page의 copy가 끝난 후에, 호출된다.

`TvpnBucket` 에 저장된 각 tvpn별로 4096bytes의 size인 `data` 에 원래 mapping을 load 한 후, 각 valid page mapping을 반영하여 `tnand_write` 를 수행한다.

Performance Analysis

Data GC의 valid page의 mapping copy batch size에 따른 성능



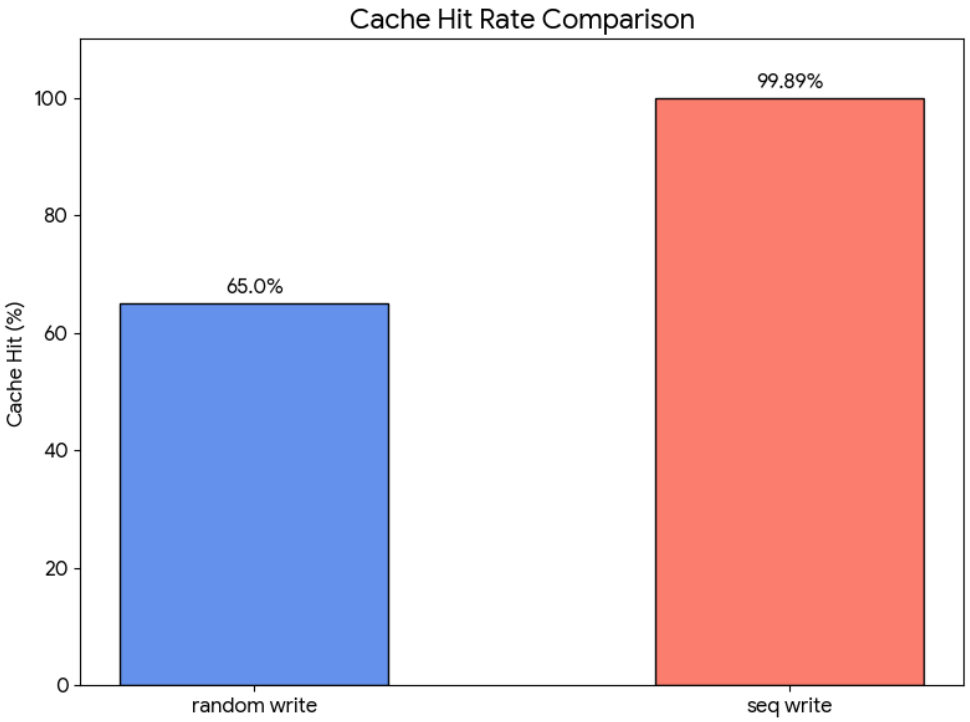
Data GC의 valid page mapping의 batch unit에 따른 memory 사용량과 nand write수의 trade-off관계를 분석하였다.

사용된 워크로드는 fio를 통해 sequential write 3G후, random write 3G loop 5이며, 파란색 히스토그램은 valid page copy에서 mapping update시 발생하는 `tnand_write()` 횟수를 정규화한 분포이고 붉은색 히스토그램은 batch를 위한 mapping을 저장하기 위해 사용되는 메모리 사용량이다.

제출된 코드에는 block단위 batch update를 구현하였으며, 비교군으로 메모리를 32배 더 사용하는 superblock 단위 batch, 기존 CDFTL논문의 알고리즘인 Linear update를 두었다.

- 기존 Linear Update 대비 block 단위 batch는 nand_write call을 약 18,000회 절약하는 결과를 보인다.
- superblock 단위 batch의 경우 약 30,000회 nand_write call을 절약하지만 block batch의 32배인 약 192KB의 메모리를 요구한다.
    - Blocks per Line = 4 * 8 (channels * luns per ch)
    - Pages per Line = 4 * 8 * 256 = 8192
    - Per Bucket size = 24 bytes
    - Bucket size = 8192 * 24 bytes = 192KB

Workload 특성에 따른 Cache Hit Rate

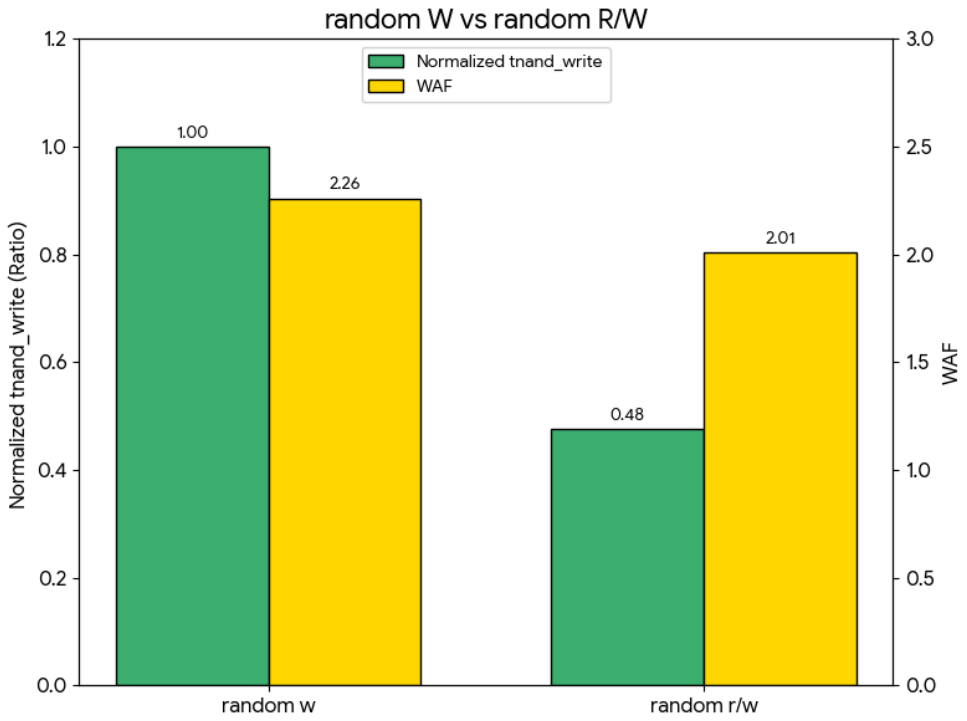


- 좌측 히스토그램 : sequential write 3G 후, rand w 3G loop5
- 우측 히스토그램 : sequential write 3G 후, seq w 3G loop5

CDFTL의 구현은 CTP를 통해 translation page를 캐싱하여, CMT로부터는 Temporal locality, CTP로부터 Spatial locality를 이용한 성능향상을 목표로 한다.

그럼에도 불과하고, Sequential write 워크로드 대비 Random Write 워크로드에서 Cache Hit rate가 약 30% 감소함을 확인할 수 있다.

Workload 특성에 따른 Write-Amplification-Factor(WAF)



- 좌측 히스토그램 : sequential write 3G 후, rand w 3G loop5
- 우측 히스토그램 : sequential write 3G → rand rw 3G loop5, (read:write비율 = 50:50)

Write I/O는 l2p mapping update를 동반하는 연산이기 때문에, write intensive한 환경대비, read/write 고루 분포되는 환경에서 WAF가 감소함을 확인할 수 있다.

Raw Stats

Batch GC(Line)

- Blocks per Line = 4 * 8 (channels * luns per ch)
- Pages per Line = 4 * 8 * 256 = 8192
- Per Bucket size = 24 bytes
- Bucket size = 8192 * 24 bytes = 192KB
  - sequential write 3G → random write 3G loop 5

```
[sudo] password for femu:
[FEMU] Log: Cache Hit Rate Log:
CMT Hits=3850208,
CTP Hits=7158651,
Cache Misses=3853269,
Total Requests=11011920,
CMT Hit Rate=34.96%,
CTP Hit Rate=65.01%,
Cache Miss Rate=34.99%
[FEMU] Log: tnand_read count : 4716792
[FEMU] Log: tnand_write count : 4710973
[FEMU] Log: tnand_write_gc count : 379534
waf: 2.180000
```

Batch GC(Block)

Pages per block = 256

Per Bucket size = 24 bytes

Bucket size = 256 * 24 bytes = 6KB

- sequential write 3G → random write 3G loop 5

```
[FEMU] Log: Cache Hit Rate Log:
CMT Hits=3850466,
CTP Hits=7158391,
Cache Misses=3853529,
Total Requests=11011920,
CMT Hit Rate=34.97%,
CTP Hit Rate=65.01%,
Cache Miss Rate=34.99%
[FEMU] Log: tnand_read count : 5097304
[FEMU] Log: tnand_write count : 5091726
[FEMU] Log: tnand_write_gc count : 674280
waf: 2.260000
```

- sequential write 3G → seq w 3G loop5

```
[FEMU] Log: Cache Hit Rate Log:
CMT Hits=9243,
CTP Hits=11000758,
Cache Misses=12299,
Total Requests=11013057,
CMT Hit Rate=0.08%,
CTP Hit Rate=99.89%,
Cache Miss Rate=0.11%
[FEMU] Log: tnand_read count : 7812
[FEMU] Log: tnand_write count : 9313
[FEMU] Log: tnand_write_gc count : 128
waf: 1.000000
```

- sequential write 3G → rand w 3G loop5

```
[FEMU] Log: Cache Hit Rate Log:
CMT Hits=3850466,
CTP Hits=7158391,
Cache Misses=3853529,
Total Requests=11011920,
CMT Hit Rate=34.97%,
CTP Hit Rate=65.01%,
Cache Miss Rate=34.99%
[FEMU] Log: tnand_read count : 5097304
[FEMU] Log: tnand_write count : 5091726
[FEMU] Log: tnand_write_gc count : 674280
waf: 2.260000
```

- sequential write 3G → seq rw 3G loop5

```
[FEMU] Log: Cache Hit Rate Log:
CMT Hits=4979,
CTP Hits=9038221,
Cache Misses=8460,
Total Requests=9046681,
CMT Hit Rate=0.06%,
CTP Hit Rate=99.91%,
Cache Miss Rate=0.09%
[FEMU] Log: tnand_read count : 3911
[FEMU] Log: tnand_write count : 5412
[FEMU] Log: tnand_write_gc count : 64
waf: 1.000000
```

- sequential write 3G → rand rw 3G loop5

```
[FEMU] Log: Cache Hit Rate Log:
CMT Hits=1926221,
CTP Hits=5194448,
Cache Misses=3852381,
Total Requests=9046829,
CMT Hit Rate=21.29%,
CTP Hit Rate=57.42%,
Cache Miss Rate=42.58%
[FEMU] Log: tnand_read count : 4319673
[FEMU] Log: tnand_write count : 2419672
[FEMU] Log: tnand_write_gc count : 194660
waf: 2.010000
```

Linear GC

- sequential write 3G → random write 3G looop 5

```
CMT Hits=3850544,
CTP Hits=7158314,
Cache Misses=3853606,
Total Requests=11011920,
CMT Hit Rate=34.97%,
CTP Hit Rate=65.01%,
Cache Miss Rate=34.99%
[FEMU] Log: tnand_read count : 5311396
[FEMU] Log: tnand_write count : 5314417
[FEMU] Log: tnand_write_gc count : 858301
```